GitHub Actions — Category 7: CI/CD for Real Workloads

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Categories 1–6 assumed. Docker, Kubernetes, Terraform, and cloud provider experience assumed. **Format per topic:** What → Why → How Internally → Key Concepts → Interview Answers → Full YAML → Gotchas ⚠ → Connections

---

Table of Contents

- 7.1 Docker Builds — buildx, Multi-Arch, Layer Caching
- 7.2 GitHub Container Registry (GHCR) and GitHub Packages
- 7.3 Kubernetes Deployments from GitHub Actions
- 7.4 Terraform Workflows — Plan, Apply, Drift Detection
- 7.5 Cloud Deployments — AWS, GCP, Azure
- 7.6 GitOps with ArgoCD — Image Updater, PR-Based Promotions
- 7.7 Blue-Green and Canary Deployment Patterns
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

7.1 Docker Builds — buildx, Multi-Arch, Layer Caching

What It Is

Docker buildx is the next-generation builder for Docker that enables multi-platform builds (building for ARM64, AMD64, etc. from a single machine), advanced layer caching backends, and BuildKit features. In GitHub Actions, it's the standard approach for all non-trivial container builds.

Jenkins equivalent: A Jenkins pipeline with a Dockerfile build stage — typically `docker build && docker push`. The difference: buildx adds multi-architecture support and proper cache management that Jenkins users typically bolt on awkwardly with external tools.

Why buildx Over Plain docker build

```
docker build (legacy):
  - Single architecture only (builds for host arch)
  - Basic layer cache: only re-uses local filesystem cache
  - No BuildKit features (no secret mounts, no SSH agent mounts)
  - No cross-platform support without QEMU/binfmt setup
```

```
docker buildx (BuildKit):
  - Multi-platform: build linux/amd64 AND linux/arm64 in one command
  - Cache: --cache-from/--cache-to with multiple backends (registry,
GitHub cache, S3)
  - Secrets: --secret id=mysecret,src=file (secret not baked into image)
  - SSH: --ssh default (SSH agent forwarded, not exposed in layer)
  - Attestations: SLSA provenance, SBOM generation
  - Build output: load (local), push (registry), or multiple destinations
```

The Standard Docker Build Workflow

```yaml
name: Docker Build and Push

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read
  packages: write       # For GHCR push
  id-token: write       # For OIDC if also pushing to ECR/GCR
  attestations: write   # For SLSA provenance

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      image-digest: ${{ steps.build.outputs.digest }}
      image-tags: ${{ steps.meta.outputs.tags }}

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2

      # Step 1: Generate semantic image tags and labels
      - name: Docker metadata
        id: meta
        uses: docker/metadata-
action@902fa8ec7d6ecbea8a986c160c70f1a6b9d84432  # v5.7.0
        with:
          images: |
            ghcr.io/${{ github.repository }}
          tags: |
            # Tag with branch name (main -> :latest, feature/x ->
:feature-x)
            type=ref,event=branch
            # Tag with PR number for PR builds: pr-123
            type=ref,event=pr
```

```
            # Semantic versioning from git tags: v1.2.3 -> 1.2.3, 1.2, 1
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=semver,pattern={{major}}
            # Always tag with short SHA for traceability
            type=sha,prefix=sha-
            # Tag 'latest' only on main branch
            type=raw,value=latest,enable={{is_default_branch}}
          labels: |
            org.opencontainers.image.title=My Application
            org.opencontainers.image.vendor=my-org

      # Step 2: Set up QEMU for multi-arch emulation
      - name: Set up QEMU
        uses: docker/setup-qemu-
action@29109295f81e9208d7d86ff9f396b2302d0b52e5  # v3.3.0
        with:
          platforms: linux/amd64,linux/arm64

      # Step 3: Set up Buildx builder
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-
action@b5730b5c3c4c89bdf3a737c8af1c4fc46a07e21d  # v3.10.0
        with:
          # Use docker-container driver for full BuildKit support
          driver: docker-container
          # Optional: use a remote BuildKit instance for faster builds
          # driver-opts: endpoint=tcp://buildkitd.internal:1234

      # Step 4: Log in to registry
      - name: Log in to GHCR
        uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
# v3.3.0
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      # Step 5: Build and push
      - name: Build and push
        id: build
        uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4  # v6.15.0
        with:
          context: .
          platforms: linux/amd64,linux/arm64
          push: ${{ github.event_name != 'pull_request' }}  # Only push on
non-PR
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          # GitHub Actions cache backend -- layer caching between runs
          cache-from: type=gha
          cache-to: type=gha,mode=max
          # Build arguments
```

```
          build-args: |
            VERSION=${{ github.sha }}
            BUILD_DATE=${{ github.event.head_commit.timestamp }}
          # Provenance and SBOM (supply chain security)
          provenance: mode=max     # SLSA level 3 provenance
          sbom: true               # Generate SBOM and attach to image

      # Step 6: Generate SLSA attestation
      - name: Attest build provenance
        uses: actions/attest-build-
provenance@db473fdca9a4a4a8de990c60c3e64c29f0e00e6f  # v2.3.0
        if: github.event_name != 'pull_request'
        with:
          subject-name: ghcr.io/${{ github.repository }}
          subject-digest: ${{ steps.build.outputs.digest }}
          push-to-registry: true
```

Multi-Architecture Builds — Deep Dive

```
# Two strategies for multi-arch:
# Strategy 1: QEMU emulation (simple, slower)
# Strategy 2: Native builders (complex setup, fast)

# Strategy 1: QEMU -- builds ARM64 image on x86 runner via emulation
# Good for: most workloads, simple setup
# Bad for: computationally intensive builds (emulation is 10-20x slower)

jobs:
  build-multiarch-qemu:
    runs-on: ubuntu-latest   # x86 runner
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: docker/setup-qemu-
action@29109295f81e9208d7d86ff9f396b2302d0b52e5
        with:
          platforms: all     # Register all QEMU binfmt handlers
      - uses: docker/setup-buildx-
action@b5730b5c3c4c89bdf3a737c8af1c4fc46a07e21d
      - uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          platforms: linux/amd64,linux/arm64,linux/arm/v7
          push: true
          tags: ghcr.io/${{ github.repository }}:latest
          cache-from: type=gha
```

```yaml
          cache-to: type=gha,mode=max

# Strategy 2: Native multi-arch -- separate jobs per architecture, then
merge
# Good for: Rust, Go compilation-heavy builds (native is much faster)
# Requires: ARM64 runner (GitHub-hosted or self-hosted)

jobs:
  build-amd64:
    runs-on: ubuntu-latest           # x86 native
    outputs:
      digest: ${{ steps.build.outputs.digest }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: docker/setup-buildx-
action@b5730b5c3c4c89bdf3a737c8af1c4fc46a07e21d
      - uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - id: build
        uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          platforms: linux/amd64
          push: true
          tags: ghcr.io/${{ github.repository }}:${{ github.sha }}-amd64
          cache-from: type=gha,scope=amd64
          cache-to: type=gha,scope=amd64,mode=max

  build-arm64:
    runs-on: ubuntu-arm-4-cores      # ARM64 native runner (GitHub-hosted
larger)
    outputs:
      digest: ${{ steps.build.outputs.digest }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: docker/setup-buildx-
action@b5730b5c3c4c89bdf3a737c8af1c4fc46a07e21d
      - uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - id: build
        uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          platforms: linux/arm64
          push: true
          tags: ghcr.io/${{ github.repository }}:${{ github.sha }}-arm64
          cache-from: type=gha,scope=arm64
          cache-to: type=gha,scope=arm64,mode=max
```

```
  # Merge separate arch manifests into a single multi-arch manifest
  merge-manifest:
    needs: [build-amd64, build-arm64]
    runs-on: ubuntu-latest
    steps:
      - uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/metadata-
action@902fa8ec7d6ecbea8a986c160c70f1a6b9d84432
        id: meta
        with:
          images: ghcr.io/${{ github.repository }}
          tags: type=sha,prefix=sha-
      - uses: int128/docker-manifest-create-action@v2
        with:
          tags: ${{ steps.meta.outputs.tags }}
          suffixes: |
            -amd64
            -arm64
```

Layer Caching Strategies

```
# Three caching backends for Docker builds:

# 1. GitHub Actions cache (type=gha) -- most common
#    Pros: free, integrated, no setup
#    Cons: 10GB repo limit, 7-day TTL, slow for large images
- uses: docker/build-push-action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
  with:
    cache-from: type=gha
    cache-to: type=gha,mode=max  # mode=max caches ALL layers (not just
final stage)
                                 # mode=min caches only final stage
layers

# 2. Registry cache (type=registry) -- best for large images
#    Pros: no size limit (beyond registry storage), longer TTL
#    Cons: registry storage cost, requires push access
- uses: docker/build-push-action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
  with:
    cache-from: type=registry,ref=ghcr.io/my-org/myapp:buildcache
    cache-to: type=registry,ref=ghcr.io/my-
org/myapp:buildcache,mode=max,image-manifest=true

# 3. Local cache + artifacts (for self-hosted persistent runners)
#    Pros: fastest (local filesystem)
```

```
#    Cons: only works on persistent runners; ephemeral runners lose cache
- uses: docker/build-push-action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
  with:
    cache-from: type=local,src=/tmp/docker-cache
    cache-to: type=local,dest=/tmp/docker-cache,mode=max
```

BuildKit Secret Mounts — Building Without Baking Secrets Into Layers

```
# Problem: many builds need secrets at build time (npm auth token, private
packages)
# Wrong: ARG NPM_TOKEN=... -> baked into image history, visible in layers
# Right: BuildKit secret mounts -- available at build time, NOT in final
image

- name: Build with secret (never baked into image)
  uses: docker/build-push-action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
  with:
    secrets: |
      npm_token=${{ secrets.NPM_TOKEN }}
    secret-envs: |
      GITHUB_TOKEN=${{ secrets.GITHUB_TOKEN }}
```

```
# Dockerfile -- using the mounted secret
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./

# Secret is available ONLY during this RUN step, not in image
RUN --mount=type=secret,id=npm_token \
    NPM_TOKEN=$(cat /run/secrets/npm_token) \
    npm config set //registry.npmjs.org/:_authToken=$NPM_TOKEN && \
    npm ci --only=production

COPY . .
RUN npm run build

# Final stage -- npm token never appears here
FROM node:20-alpine AS runtime
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
CMD ["node", "dist/server.js"]
```

Interview Answer (30-45 seconds)

> "Docker buildx with the BuildKit backend is the standard for GitHub Actions container builds. The key components: `setup-qemu-action` registers QEMU binfmt handlers for cross-arch emulation, `setup-buildx-action` creates a BuildKit builder instance, `metadata-action` generates semantic tags from git refs (SHA, branches, semver from tags), and `build-push-action` executes the actual build with cache configuration. For layer caching, `type=gha` stores layers in GitHub's cache with `mode=max` to cache intermediate stages too. Multi-arch builds have two strategies: QEMU emulation (simple, 10-20x slower for compiled code) or native parallel builds (separate jobs per arch, merged with a manifest list). For build-time secrets like npm tokens, use BuildKit secret mounts so the token is available during `RUN` but never baked into image layers or visible in image history."

Gotchas ⚠

- **`push: ${{ github.event_name != 'pull_request' }}` is the standard PR safety.** On PRs you want to build (to catch errors) but not push (to avoid polluting the registry with PR images). The boolean expression evaluates correctly.
- **QEMU emulation for ARM64 Rust or Go builds can take 10-30x longer** than native. A 3-minute native build becomes a 30-60 minute emulated build. Always use native ARM64 runners for compiled language projects.
- **`mode=max` vs `mode=min` in cache-to matters a lot.** `mode=min` only caches the final stage — intermediate stages of multi-stage Dockerfiles are NOT cached. For a build like `go build` in an intermediate stage, `mode=max` is required to get cache hits on the compilation stage.
- **Buildx requires the `docker-container` driver for multi-platform.** The default `docker` driver does NOT support multi-platform builds. Always set `docker/setup-buildx-action` explicitly; don't assume the default Docker daemon supports buildx features.
- **Registry cache requires `image-manifest=true` for OCI-compliant registries** including GHCR. Without this flag, cache writes fail on modern registry APIs.
- **`metadata-action` returns `tags` as a newline-separated string.** `build-push-action` accepts this directly. Don't try to parse or split it.

7.2 GitHub Container Registry (GHCR) and GitHub Packages

What It Is

GitHub Container Registry (GHCR) is GitHub's OCI-compliant container registry, available at `ghcr.io`. GitHub Packages is the broader package hosting service that includes GHCR plus npm, Maven, NuGet, RubyGems, and Gradle package registries — all integrated with GitHub's authentication model.

Why it matters operationally: GHCR uses `GITHUB_TOKEN` for auth (no separate credentials), packages can inherit repository visibility (public repo = public image by default), and access control is unified with GitHub teams and org membership.

GHCR Authentication Patterns

```
# Pattern 1: GITHUB_TOKEN (most common -- works for repo's own images)
- uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}
    # Requires workflow permission: packages: write (for push)
    # packages: read is default and sufficient for pull

# Pattern 2: Personal Access Token (for cross-repo or external access)
- uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GHCR_PAT }}   # PAT with read:packages or
write:packages scope

# Pattern 3: GitHub App token (for org-wide automation)
- uses: actions/create-github-app-
token@5d869da34e18e7287c1daad50e0b8ea0f506ce69
  id: app-token
  with:
    app-id: ${{ vars.BOT_APP_ID }}
    private-key: ${{ secrets.BOT_APP_PRIVATE_KEY }}
    owner: my-org
- uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
  with:
    registry: ghcr.io
    username: ${{ steps.app-token.outputs.app-slug }}
    password: ${{ steps.app-token.outputs.token }}
```

GHCR Image Naming Conventions

```
ghcr.io/<owner>/<image-name>:<tag>

Owner:
  - github.com user:  ghcr.io/jane-dev/myapp
  - github.com org:   ghcr.io/my-org/myapp
  - Using github.repository: ghcr.io/${{ github.repository }}
    (resolves to ghcr.io/my-org/my-repo -- repo name as image name)

For monorepos with multiple services:
  ghcr.io/my-org/payment-service:sha-abc123
  ghcr.io/my-org/user-service:sha-abc123
  ghcr.io/my-org/inventory-service:sha-abc123
```

```
Using github.repository_owner:
  ghcr.io/${{ github.repository_owner }}/payment-service:${{ github.sha }}
```

Complete GHCR Workflow with Visibility Control

```yaml
name: Build and Publish to GHCR

on:
  push:
    branches: [main]
    tags: ['v*']
  pull_request:
    branches: [main]

permissions:
  contents: read
  packages: write
  attestations: write
  id-token: write

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Generate image metadata
        id: meta
        uses: docker/metadata-
action@902fa8ec7d6ecbea8a986c160c70f1a6b9d84432
        with:
          images: ghcr.io/${{ github.repository_owner }}/myapp
          tags: |
            type=sha,prefix=sha-
            type=semver,pattern={{version}}
            type=ref,event=branch
            type=raw,value=latest,enable=${{ github.ref ==
'refs/heads/main' }}

      - uses: docker/setup-buildx-
action@b5730b5c3c4c89bdf3a737c8af1c4fc46a07e21d

      - uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
        if: github.event_name != 'pull_request'
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push to GHCR
```

```
        id: push
        uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          context: .
          platforms: linux/amd64,linux/arm64
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Attest provenance
        uses: actions/attest-build-
provenance@db473fdca9a4a4a8de990c60c3e64c29f0e00e6f
        if: github.event_name != 'pull_request'
        with:
          subject-name: ghcr.io/${{ github.repository_owner }}/myapp
          subject-digest: ${{ steps.push.outputs.digest }}

  # Control GHCR package visibility after push
  set-package-visibility:
    needs: build-and-push
    runs-on: ubuntu-latest
    if: github.event_name != 'pull_request'
    steps:
      - name: Set package to private
        run: |
          # GHCR packages default to private for org repos
          # Explicitly set visibility via API
          gh api \
            --method PATCH \
            /orgs/my-org/packages/container/myapp \
            -f visibility=private
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

GitHub Packages for Non-Container Artifacts

```
# Publishing npm package to GitHub Packages
name: Publish npm Package

on:
  push:
    tags: ['v*']

permissions:
  contents: read
  packages: write
```

```yaml
jobs:
  publish-npm:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
        with:
          node-version: '20'
          registry-url: 'https://npm.pkg.github.com'
          scope: '@my-org'    # npm scope must match GitHub org

      - run: npm ci
      - run: npm test
      - run: npm publish
        env:
          NODE_AUTH_TOKEN: ${{ secrets.GITHUB_TOKEN }}

# Consuming GitHub Packages npm from other repos
# .npmrc in consuming repo:
# @my-org:registry=https://npm.pkg.github.com
# //npm.pkg.github.com/:_authToken=${GITHUB_TOKEN}
```

Interview Answer (30-45 seconds)

> "GHCR is GitHub's OCI-compliant container registry at `ghcr.io`. The key operational advantage is the authentication model: `GITHUB_TOKEN` with `packages: write` permission authenticates directly, no separate registry credentials needed. Image names follow `ghcr.io/<owner>/<image>:<tag>` where owner is the GitHub user or org. Packages inherit visibility from the linked repository by default — private repos get private packages. For multi-service repos, use `ghcr.io/${{ github.repository_owner }}/service-name` to keep images in the org namespace. GHCR also supports provenance attestations via `actions/attest-build-provenance`, which is increasingly required for supply chain compliance. For non-container packages, GitHub Packages supports npm, Maven, NuGet, RubyGems, and Gradle using the same `GITHUB_TOKEN` auth pattern."

Gotchas ⚠

- **Package visibility defaults are counter-intuitive.** A package pushed from a public repo is public by default. A package pushed from a private repo is private. But packages are independently managed — deleting or changing the repo doesn't automatically affect the package visibility.
- **`packages: write` permission is required for push, but `packages: read` is default.** If you don't explicitly add `packages: write` to your workflow permissions, the login succeeds but the push fails with an authorization error.
- **GHCR uses the repository's linked package concept.** When you push to `ghcr.io/my-org/my-repo`, GitHub links that package to the `my-org/my-repo` repository. The package appears in the repo's "Packages" sidebar. Access control follows the repo's collaborator settings.

- **The `github.actor` username for GHCR login must match exactly.** If a bot or app token is used, the username must be the app's login (e.g., `my-bot[bot]`), not a human username.

7.3 Kubernetes Deployments from GitHub Actions

What It Is

Patterns for deploying applications to Kubernetes clusters from GitHub Actions workflows — covering kubectl direct deployments, Helm chart deployments, and the integration patterns for different cloud providers' managed Kubernetes offerings.

The Fundamental Problem: Cluster Authentication

```
GitHub Actions runner (public internet or VPC)
        |
        | needs to reach:
        v
Kubernetes API server (may be private or public)
        |
        v
Options for authentication:
  1. OIDC + cloud provider IAM -> kubeconfig via AWS/GCP/Azure CLI
  2. Service account token in GitHub Secret (static, rotates manually)
  3. VPN/private link from runner VPC to cluster VPC
  4. Self-hosted runner inside the cluster VPC
```

Pattern 1: kubectl Direct Deployment (Simple Services)

```yaml
name: Deploy to Kubernetes

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
```

```
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      # Authenticate to cloud provider via OIDC (see Cat 4.3)
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.PROD_DEPLOY_ROLE_ARN }}
          aws-region: us-east-1

      # Get kubeconfig from EKS
      - name: Configure kubectl
        run: |
          aws eks update-kubeconfig \
            --name prod-cluster \
            --region us-east-1 \
            --kubeconfig /tmp/kubeconfig

      # Apply Kubernetes manifests
      - name: Apply manifests
        run: |
          kubectl apply -f k8s/namespace.yaml \
            --kubeconfig /tmp/kubeconfig
          kubectl apply -f k8s/ \
            --kubeconfig /tmp/kubeconfig \
            --recursive

      # Update the image tag in the deployment
      - name: Update image
        run: |
          kubectl set image deployment/myapp \
            app=ghcr.io/my-org/myapp:sha-${{ github.sha }} \
            --namespace production \
            --kubeconfig /tmp/kubeconfig

      # Wait for rollout to complete
      - name: Wait for rollout
        run: |
          kubectl rollout status deployment/myapp \
            --namespace production \
            --timeout=5m \
            --kubeconfig /tmp/kubeconfig

      # Verify deployment health
      - name: Verify deployment
        run: |
          READY=$(kubectl get deployment myapp \
            --namespace production \
            --kubeconfig /tmp/kubeconfig \
            -o jsonpath='{.status.readyReplicas}')
          DESIRED=$(kubectl get deployment myapp \
            --namespace production \
            --kubeconfig /tmp/kubeconfig \
            -o jsonpath='{.spec.replicas}')
```

```
          echo "Ready: $READY / Desired: $DESIRED"
          [ "$READY" = "$DESIRED" ] || (echo "Deployment not healthy" &&
exit 1)
```

Pattern 2: Helm Deployment (Production Standard)

```yaml
name: Helm Deploy

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  helm-deploy:
    runs-on: ubuntu-latest
    environment: production
    outputs:
      release-revision: ${{ steps.deploy.outputs.revision }}

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.PROD_DEPLOY_ROLE_ARN }}
          aws-region: us-east-1

      - name: Install Helm
        uses: azure/setup-helm@b9e51907a09c216f16ebe8536097933489208112  #
v4
        with:
          version: '3.14.0'

      - name: Configure kubectl
        run: aws eks update-kubeconfig --name prod-cluster --region us-
east-1

      - name: Add Helm repos
        run: |
          helm repo add stable https://charts.helm.sh/stable
          helm repo update

      - name: Helm deploy
        id: deploy
```

```
        run: |
          helm upgrade --install myapp ./helm/myapp \
            --namespace production \
            --create-namespace \
            --values ./helm/myapp/values.yaml \
            --values ./helm/myapp/values-production.yaml \
            --set image.tag=sha-${{ github.sha }} \
            --set image.repository=ghcr.io/my-org/myapp \
            --set replicaCount=3 \
            --set ingress.host=app.my-org.com \
            --set podAnnotations."deploy/sha"=${{ github.sha }} \
            --wait \
            --timeout 10m \
            --atomic          # Rollback automatically if deploy fails
            # --dry-run       # Add for testing

          REVISION=$(helm history myapp --namespace production --max 1 \
            --output json | jq -r '.[0].revision')
          echo "revision=${REVISION}" >> $GITHUB_OUTPUT
          echo "Deployed Helm revision: $REVISION"

      - name: Run post-deploy smoke tests
        run: |
          # Run smoke tests against the freshly deployed service
          kubectl run smoke-test \
            --image=ghcr.io/my-org/smoke-tests:latest \
            --rm \
            --attach \
            --restart=Never \
            --env="TARGET_URL=https://app.my-org.com" \
            --namespace=production \
            -- /smoke-tests.sh

      - name: Helm rollback on failure
        if: failure() && steps.deploy.outcome != 'skipped'
        run: |
          echo "Deploy failed -- rolling back to previous revision"
          helm rollback myapp \
            --namespace production \
            --wait \
            --timeout 5m
```

Pattern 3: Kustomize Deployment

```
# Kustomize is ideal when you have base manifests + per-environment
overlays

# Directory structure:
# k8s/
#   base/
```

```
#     deployment.yaml
#     service.yaml
#     kustomization.yaml
#   overlays/
#     staging/
#       kustomization.yaml   (patches image tag, replicas, env vars)
#     production/
#       kustomization.yaml

jobs:
  kustomize-deploy:
    runs-on: ubuntu-latest
    environment: production
    strategy:
      matrix:
        env: [staging, production]
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Setup Kustomize
        uses: imranismail/setup-
kustomize@a76db1c6419124d51470b1e388c4b29476f495f1  # v2

      - name: Update image tag in overlay
        working-directory: k8s/overlays/${{ matrix.env }}
        run: |
          kustomize edit set image \
            ghcr.io/my-org/myapp=ghcr.io/my-org/myapp:sha-${{ github.sha
}}

      - name: Build and apply manifests
        run: |
          kustomize build k8s/overlays/${{ matrix.env }} \
            | kubectl apply -f - \
              --kubeconfig /tmp/kubeconfig-${{ matrix.env }}

      - name: Wait for rollout
        run: |
          kubectl rollout status deployment/myapp \
            --namespace ${{ matrix.env }} \
            --timeout=5m
```

Pattern 4: Kubeconfig via Secret (When OIDC Is Not Available)

```
# Last resort: base64-encoded kubeconfig stored as GitHub Secret
# Requires rotation management -- not recommended for production

jobs:
  deploy-fallback:
    runs-on: ubuntu-latest
```

```
    steps:
      - name: Configure kubectl from secret
        run: |
          mkdir -p ~/.kube
          echo "${{ secrets.KUBECONFIG_BASE64 }}" | base64 -d >
~/.kube/config
          chmod 600 ~/.kube/config
          # Verify connectivity
          kubectl cluster-info

      - run: kubectl apply -f k8s/
```

Kubernetes Deployment Rollout Verification

```
# Comprehensive post-deploy verification
- name: Post-deploy verification
  run: |
    NAMESPACE="production"
    DEPLOYMENT="myapp"

    # Wait for rollout
    kubectl rollout status deployment/$DEPLOYMENT \
      --namespace $NAMESPACE \
      --timeout=10m

    # Check all pods are running
    TOTAL=$(kubectl get deployment $DEPLOYMENT -n $NAMESPACE \
      -o jsonpath='{.spec.replicas}')
    READY=$(kubectl get deployment $DEPLOYMENT -n $NAMESPACE \
      -o jsonpath='{.status.readyReplicas}')
    UPDATED=$(kubectl get deployment $DEPLOYMENT -n $NAMESPACE \
      -o jsonpath='{.status.updatedReplicas}')

    echo "Total: $TOTAL, Ready: $READY, Updated: $UPDATED"

    if [ "$READY" != "$TOTAL" ] || [ "$UPDATED" != "$TOTAL" ]; then
      echo "Deployment not healthy -- dumping pod logs"
      kubectl get pods -n $NAMESPACE -l app=$DEPLOYMENT
      kubectl describe deployment $DEPLOYMENT -n $NAMESPACE
      # Get logs from any failing pods
      kubectl get pods -n $NAMESPACE -l app=$DEPLOYMENT \
        -o jsonpath='{.items[*].metadata.name}' | \
        tr ' ' '\n' | \
        xargs -I{} kubectl logs {} -n $NAMESPACE --tail=50 || true
      exit 1
    fi

    # HTTP health check
    SVC_IP=$(kubectl get svc $DEPLOYMENT -n $NAMESPACE \
      -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
```

```
      for i in {1..10}; do
        STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
          http://$SVC_IP/health --max-time 5)
        [ "$STATUS" = "200" ] && echo "Health check passed" && exit 0
        echo "Attempt $i: HTTP $STATUS"
        sleep 10
      done
      echo "Health check failed after 10 attempts"
      exit 1
```

Interview Answer (30-45 seconds)

> "Kubernetes deployments from GitHub Actions have two key concerns: cluster authentication and rollout verification. For authentication, OIDC is the right approach — authenticate to the cloud provider, then use the provider's CLI to generate a kubeconfig (`aws eks update-kubeconfig`, `gcloud container clusters get-credentials`). For the actual deployment, Helm is the production standard: `helm upgrade --install` with `--atomic` provides automatic rollback if the deploy fails, and `--wait --timeout` ensures you don't declare success before pods are actually healthy. After deploy, always verify with `kubectl rollout status` and a health check HTTP probe — `helm --atomic` only checks Kubernetes readiness probes, not application-level health. For GitOps workflows, you don't deploy directly at all — you update the image tag in a git repo and let ArgoCD reconcile the cluster state."

Gotchas ⚠

- **`helm upgrade --atomic` rolls back on failure but the workflow job still fails.** The cluster is left in the previous good state, but you still need to investigate why the new deploy failed. `atomic` is for protecting the cluster, not for silencing failures.
- **`kubectl rollout status` times out but doesn't necessarily fail the workflow.** If you use `--timeout=5m` and the rollout isn't complete, it exits with non-zero BUT the Kubernetes rollout continues. The old pods may still be running. Always verify pod health after the timeout.
- **kubeconfig files contain sensitive credentials.** Writing them to the home directory (`.kube/config`) on a persistent runner means subsequent jobs can read them. Write to `/tmp/kubeconfig` and pass explicitly via `--kubeconfig`, or ensure cleanup.
- **Private EKS/GKE clusters need runner network access.** If the cluster API server is not publicly accessible, the runner must be in the same VPC (self-hosted runner) or connected via private endpoint. GitHub-hosted runners cannot reach private cluster API servers.
- **Image pull secrets must pre-exist in the target namespace.** If your cluster needs to pull from a private registry, the imagePullSecret must already be configured in Kubernetes before the deploy. Don't create it in the workflow — manage it with cluster bootstrap tooling.

7.4 Terraform Workflows — Plan, Apply, Drift Detection

What It Is

Patterns for managing Terraform (and OpenTofu) infrastructure changes through GitHub Actions — including PR-based plan preview, protected apply on merge, drift detection, and the architectural decision between GitHub Actions-native Terraform workflows and Atlantis.

The Core Terraform Workflow Pattern

```
PR opened/updated:
  terraform init
  terraform validate
  terraform plan  -> Post plan output as PR comment
  (human reviews plan in PR comment, then approves PR)

PR merged to main:
  terraform init
  terraform plan  (verify no drift since PR)
  terraform apply

Nightly:
  terraform plan  -> If output != "No changes", alert (drift detection)
```

Full PR + Apply Workflow

```yaml
name: Terraform

on:
  push:
    branches: [main]
    paths:
      - 'terraform/**'
      - '.github/workflows/terraform.yml'
  pull_request:
    branches: [main]
    paths:
      - 'terraform/**'

permissions:
  id-token: write
  contents: read
  pull-requests: write    # For posting plan as PR comment

env:
  TF_VERSION: "1.7.0"
  AWS_REGION: "us-east-1"
  TF_WORKING_DIR: "./terraform"
```

```yaml
jobs:

  terraform-plan:
    name: Terraform Plan
    runs-on: ubuntu-latest
    outputs:
      plan-exitcode: ${{ steps.plan.outputs.exitcode }}
    defaults:
      run:
        working-directory: ${{ env.TF_WORKING_DIR }}

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      # OIDC auth to AWS
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.TF_PLAN_ROLE_ARN }}  # Read-only
role for plans
          aws-region: ${{ env.AWS_REGION }}

      - uses: hashicorp/setup-
terraform@b9cd54a3c349d3f38e8881555d616ced269ef032  # v3.1.2
        with:
          terraform_version: ${{ env.TF_VERSION }}
          cli_config_credentials_token: ${{ secrets.TF_CLOUD_TOKEN }}  #
If using TFC

      - name: Terraform Init
        id: init
        run: terraform init -backend-
config="key=production/terraform.tfstate"

      - name: Terraform Validate
        id: validate
        run: terraform validate -no-color

      - name: Terraform Format Check
        id: fmt
        run: terraform fmt -check -recursive -no-color
        continue-on-error: true    # Don't fail -- just report

      - name: Terraform Plan
        id: plan
        run: |
          terraform plan \
            -no-color \
            -detailed-exitcode \
            -out=tfplan \
            -input=false \
            -var-file="environments/production.tfvars"
          echo "exitcode=$?" >> $GITHUB_OUTPUT
        # Exit codes: 0=no changes, 1=error, 2=changes present
```

```
        continue-on-error: true    # Capture exit code before checking

      # Save plan for apply job
      - name: Save plan artifact
        if: steps.plan.outcome != 'failure'
        uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
        with:
          name: tfplan-${{ github.sha }}
          path: ${{ env.TF_WORKING_DIR }}/tfplan
          retention-days: 5

      # Post plan to PR as comment
      - name: Post plan to PR
        if: github.event_name == 'pull_request'
        uses: actions/github-
script@60a0d83039c74a4aee543508d2ffcb1c3799cdea  # v7
        env:
          PLAN: ${{ steps.plan.outputs.stdout }}
        with:
          script: |
            const output = `## Terraform Plan 📝

            | Step | Status |
            |------|--------|
            | Init | ${{ steps.init.outcome == 'success' && '✅' || '❌'
}} |
            | Validate | ${{ steps.validate.outcome == 'success' && '✅'
|| '❌' }} |
            | Format | ${{ steps.fmt.outcome == 'success' && '✅' || '⚠
formatting issues' }} |
            | Plan | ${{ steps.plan.outcome == 'success' && '✅' || '❌'
}} |

            <details><summary>Plan output</summary>

            \`\`\`terraform
            ${process.env.PLAN}
            \`\`\`
            </details>

            *Triggered by @${{ github.actor }} on \`${{ github.ref_name
}}\`*`;

            // Find and update existing comment, or create new one
            const { data: comments } = await
github.rest.issues.listComments({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.issue.number,
            });

            const botComment = comments.find(c =>
              c.user.type === 'Bot' && c.body.includes('Terraform Plan')
            );
```

```
            if (botComment) {
              github.rest.issues.updateComment({
                owner: context.repo.owner,
                repo: context.repo.repo,
                comment_id: botComment.id,
                body: output
              });
            } else {
              github.rest.issues.createComment({
                owner: context.repo.owner,
                repo: context.repo.repo,
                issue_number: context.issue.number,
                body: output
              });
            }

      # Fail the job if plan errored
      - name: Check plan exit code
        run: |
          EXITCODE="${{ steps.plan.outputs.exitcode }}"
          if [ "$EXITCODE" = "1" ]; then
            echo "Terraform plan failed"
            exit 1
          fi

  terraform-apply:
    name: Terraform Apply
    needs: terraform-plan
    runs-on: ubuntu-latest
    if: |
      github.event_name == 'push' &&
      github.ref == 'refs/heads/main' &&
      needs.terraform-plan.outputs.plan-exitcode == '2'
      # Only apply if there are actual changes (exit code 2)
    environment: production    # Requires approval gate
    defaults:
      run:
        working-directory: ${{ env.TF_WORKING_DIR }}

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      # Apply uses a MORE PRIVILEGED role than plan
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.TF_APPLY_ROLE_ARN }}  # Write role
          aws-region: ${{ env.AWS_REGION }}

      - uses: hashicorp/setup-
terraform@b9cd54a3c349d3f38e8881555d616ced269ef032
        with:
          terraform_version: ${{ env.TF_VERSION }}
```

```
      # Download the plan artifact (ensures apply matches what was
reviewed)
      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16
        with:
          name: tfplan-${{ github.sha }}
          path: ${{ env.TF_WORKING_DIR }}

      - name: Terraform Init
        run: terraform init -backend-
config="key=production/terraform.tfstate"

      - name: Terraform Apply
        run: |
          terraform apply \
            -auto-approve \
            -input=false \
            tfplan    # Apply the EXACT plan that was reviewed
```

Drift Detection Workflow

```
name: Terraform Drift Detection

on:
  schedule:
    - cron: '0 6 * * *'    # 6 AM UTC daily
  workflow_dispatch:

permissions:
  id-token: write
  contents: read
  issues: write    # For creating drift alert issue

jobs:
  detect-drift:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        environment: [staging, production, dr]
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets[format('TF_PLAN_ROLE_{0}',
upper(matrix.environment))] }}
          aws-region: us-east-1
```

```
      - uses: hashicorp/setup-
terraform@b9cd54a3c349d3f38e8881555d616ced269ef032
        with:
          terraform_version: "1.7.0"

      - name: Init
        working-directory: terraform/environments/${{ matrix.environment
}}
        run: terraform init

      - name: Drift detection plan
        id: drift
        working-directory: terraform/environments/${{ matrix.environment
}}
        run: |
          terraform plan \
            -detailed-exitcode \
            -no-color \
            -refresh=true \
            -out=/tmp/drift-plan.tfplan \
            2>&1 | tee /tmp/drift-output.txt

          EXITCODE=$?
          echo "exitcode=$EXITCODE" >> $GITHUB_OUTPUT

          if [ $EXITCODE -eq 2 ]; then
            echo "drift-detected=true" >> $GITHUB_OUTPUT
            echo "DRIFT DETECTED in ${{ matrix.environment }}"
          elif [ $EXITCODE -eq 0 ]; then
            echo "drift-detected=false" >> $GITHUB_OUTPUT
            echo "No drift in ${{ matrix.environment }}"
          else
            echo "Plan error in ${{ matrix.environment }}"
            exit 1
          fi
        continue-on-error: true

      - name: Create drift alert issue
        if: steps.drift.outputs.drift-detected == 'true'
        uses: actions/github-
script@60a0d83039c74a4aee543508d2ffcb1c3799cdea
        with:
          script: |
            const fs = require('fs');
            const driftOutput = fs.readFileSync('/tmp/drift-output.txt',
'utf8');
            const title = `[Drift Alert] Terraform drift detected in ${{
matrix.environment }}`;

            // Check if drift issue already open
            const { data: issues } = await
github.rest.issues.listForRepo({
              owner: context.repo.owner,
```

```
                repo: context.repo.repo,
                state: 'open',
                labels: ['terraform-drift', '${{ matrix.environment }}']
              });

              const body = `## Terraform Drift Detected in \`${{
matrix.environment }}\`

              **Detected at:** ${new Date().toISOString()}
              **Workflow run:** [${context.runId}]
(https://github.com/${context.repo.owner}/${context.repo.repo}/actions/run
s/${context.runId})

              <details><summary>Drift details</summary>

              \`\`\`
              ${driftOutput.substring(0, 65000)}
              \`\`\`
              </details>

              To remediate: review the changes, then either:
              - Apply: trigger the [Terraform Apply]
(../actions/workflows/terraform-apply.yml) workflow
              - Import: run \`terraform import\` for externally created
resources
              - Remove from state: \`terraform state rm\` for intentionally
deleted resources`;

              if (issues.length === 0) {
                await github.rest.issues.create({
                  owner: context.repo.owner,
                  repo: context.repo.repo,
                  title,
                  body,
                  labels: ['terraform-drift', 'infrastructure', '${{
matrix.environment }}']
                });
              } else {
                await github.rest.issues.createComment({
                  owner: context.repo.owner,
                  repo: context.repo.repo,
                  issue_number: issues[0].number,
                  body: `Drift still present as of ${new
Date().toISOString()}\n\n${body}`
                });
              }
```

GitHub Actions vs Atlantis — Architectural Comparison

```
ATLANTIS:
  Model: Persistent server that listens for GitHub webhooks
  Plan: Triggered by PR comment "atlantis plan"
  Apply: Triggered by PR comment "atlantis apply" (after approval)
  State locking: Built-in -- Atlantis serializes plans/applies per
workspace
  VCS integration: Deep -- Atlantis manages PR status checks directly
  Infra: Requires running Atlantis server (k8s deployment or VM)
  Config: atlantis.yaml per repo
  Automerge: Can auto-merge PR after successful apply

  Best for:
    - Teams that want a dedicated GitOps tool for infra
    - Multiple workspaces per repo with complex project routing
    - Teams comfortable running additional infrastructure

GITHUB ACTIONS:
  Model: Event-driven, no persistent server
  Plan: Triggered by PR open/update via workflow
  Apply: Triggered by push to main (or workflow_dispatch with approval)
  State locking: Delegated to Terraform backend (S3+DynamoDB, TFC, etc.)
  VCS integration: Native GitHub status checks, PR comments via API
  Infra: Zero additional infrastructure
  Config: Workflow YAML files
  Automerge: Possible via github-script but non-trivial

  Best for:
    - Teams already using GitHub Actions for application CI/CD
    - Simpler workspace topologies
    - Orgs that don't want to run additional infrastructure
    - Reusing OIDC cloud auth already set up for app deployments

Decision rule:
  Complex multi-workspace Terraform with dedicated infra team -> Atlantis
  Startup/mid-size with simple topology, GitHub Actions already used ->
GHA
  Enterprise with both needs -> Atlantis for infra, GHA for apps
```

Interview Answer (30-45 seconds)

> "The standard Terraform GitHub Actions pattern separates plan and apply into two jobs with different IAM roles. On PRs, the plan job runs with a read-only role, generates a plan with `-detailed-exitcode`, and posts the plan output as a PR comment (updating rather than creating a new comment on each push). On merge to main, the apply job downloads the artifact of the exact plan that was reviewed — this is critical, it prevents 'plan then apply a different plan' race conditions — and applies it with a write-privileged role behind an environment approval gate. Drift detection runs on a cron schedule, runs `terraform plan -refresh=true`, and creates or updates a GitHub Issue if exit code 2 (changes present). The alternative is Atlantis, which is a purpose-built tool for this workflow with better multi-workspace support, but requires running additional infrastructure."

Gotchas ⚠

- **The plan artifact MUST be used for apply.** If you run `terraform plan` in the plan job and `terraform plan` again in the apply job, the state may have changed between the two runs (another infra change landed). Always save the plan binary and apply that exact artifact.
- **`terraform plan` stdout is not captured in `steps.plan.outputs.stdout` by default.** You need `hashicorp/setup-terraform` with `terraform_wrapper: true` (default) for outputs capture. With the wrapper, stdout/stderr are captured. Without it, you need `tee` to capture output.
- **Terraform state locking prevents concurrent applies.** If two Terraform apply jobs run simultaneously (possible if two PRs merge in quick succession), one will error with "state locked." Use `concurrency:` on the apply job to serialize runs.
- **`-detailed-exitcode` with `continue-on-error: true` is required.** Exit code 2 means "changes present" — not an error. Without `continue-on-error: true`, the job fails at the plan step before you can capture the exit code.
- **Sensitive values in plan output are masked in state but NOT in plan text.** `terraform plan` may print "password = (sensitive value)" but in some cases outputs sensitive values. Never post full plan output publicly; always use `<details>` collapse in PR comments.

7.5 Cloud Deployments — AWS, GCP, Azure

What It Is

Practical patterns for the most common cloud deployment targets across the three major providers — with GitHub Actions handling orchestration and OIDC handling authentication.

AWS Deployments

ECS (Elastic Container Service)

```yaml
name: Deploy to ECS

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  deploy-ecs:
    runs-on: ubuntu-latest
    environment: production
```

```yaml
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.PROD_DEPLOY_ROLE_ARN }}
          aws-region: us-east-1

      - name: Login to Amazon ECR
        id: ecr-login
        uses: aws-actions/amazon-ecr-
login@062b18b96a7aff071d4dc91bc00c4c1a7945b076

      - name: Build and push to ECR
        uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          push: true
          tags: ${{ steps.ecr-login.outputs.registry }}/myapp:${{
github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

      # Download current task definition and update image
      - name: Update ECS task definition
        id: task-def
        uses: aws-actions/amazon-ecs-render-task-
definition@3dc97618f31c3c0a2f7de0c7879e8fed3e1fe258
        with:
          task-definition: task-definition.json    # Current task def
(checked in or downloaded)
          container-name: myapp
          image: ${{ steps.ecr-login.outputs.registry }}/myapp:${{
github.sha }}
          environment-variables: |
            APP_VERSION=${{ github.sha }}
            DEPLOY_TIME=${{ github.event.head_commit.timestamp }}

      # Deploy new task definition to ECS service
      - name: Deploy to ECS service
        uses: aws-actions/amazon-ecs-deploy-task-
definition@3a83dec0acbb8e254aa5ef0a57ca93a8cb2e7d8c
        with:
          task-definition: ${{ steps.task-def.outputs.task-definition }}
          service: myapp-service
          cluster: production
          wait-for-service-stability: true
          wait-for-minutes: 10
          # Force new deployment even if task def unchanged
          force-new-deployment: true
          # Circuit breaker: rollback automatically if service fails
          enable-ecs-managed-tags: true
          propagate-tags: SERVICE
```

```
      - name: Verify ECS service health
        run: |
          aws ecs wait services-stable \
            --cluster production \
            --services myapp-service \
            --region us-east-1
          echo "ECS service is stable"
```

Lambda (Serverless)

```
name: Deploy Lambda Function

on:
  push:
    branches: [main]
    paths:
      - 'lambda/**'
      - '.github/workflows/lambda.yml'

permissions:
  id-token: write
  contents: read

jobs:
  deploy-lambda:
    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.LAMBDA_DEPLOY_ROLE_ARN }}
          aws-region: us-east-1

      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Build Lambda deployment package
        working-directory: lambda/
        run: |
          pip install -r requirements.txt -t ./package/
          cp -r src/ ./package/
          cd package && zip -r ../function.zip .

      - name: Deploy Lambda function
        run: |
```

```
          # Update function code
          aws lambda update-function-code \
            --function-name my-function \
            --zip-file fileb://lambda/function.zip \
            --region us-east-1

          # Wait for update to complete
          aws lambda wait function-updated \
            --function-name my-function

          # Update configuration if needed
          aws lambda update-function-configuration \
            --function-name my-function \
            --environment "Variables={VERSION=${{ github.sha }}}"

          # Publish new version
          VERSION=$(aws lambda publish-version \
            --function-name my-function \
            --description "sha-${{ github.sha }}" \
            --query 'Version' \
            --output text)

          # Update alias to point to new version
          aws lambda update-alias \
            --function-name my-function \
            --name production \
            --function-version $VERSION

          echo "Deployed Lambda version: $VERSION"

      - name: Test Lambda deployment
        run: |
          RESPONSE=$(aws lambda invoke \
            --function-name my-function:production \
            --payload '{"action":"healthcheck"}' \
            --cli-binary-format raw-in-base64-out \
            /tmp/response.json \
            --query 'StatusCode' \
            --output text)

          echo "Lambda response code: $RESPONSE"
          cat /tmp/response.json

          [ "$RESPONSE" = "200" ] || (echo "Lambda healthcheck failed" &&
exit 1)
```

GCP Deployments

Cloud Run

```yaml
name: Deploy to Cloud Run

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  deploy-cloud-run:
    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: google-github-actions/auth@71fee32a0bb7e97b4d33d548e7d957010649d8fa
        with:
          workload_identity_provider: ${{ secrets.GCP_WORKLOAD_IDENTITY_PROVIDER }}
          service_account: ${{ secrets.GCP_SERVICE_ACCOUNT }}

      - uses: google-github-actions/setup-gcloud@6189d56e4096ee891640a2f1a290d3d36a88b36

      - name: Configure Docker for GCR
        run: gcloud auth configure-docker us-central1-docker.pkg.dev

      - name: Build and push to Artifact Registry
        uses: docker/build-push-action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          push: true
          tags: us-central1-docker.pkg.dev/my-project/my-repo/myapp:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Deploy to Cloud Run
        uses: google-github-actions/deploy-cloudrun@d1e31ae5e6c7e9b01b01ac2c2d99d4b7b05d3f1e
        with:
          service: myapp
          region: us-central1
          image: us-central1-docker.pkg.dev/my-project/my-repo/myapp:${{ github.sha }}
          flags: |
            --min-instances=1
            --max-instances=100
            --cpu=2
```

```
            --memory=2Gi
            --concurrency=80
            --timeout=30s
            --set-env-vars=VERSION=${{ github.sha }}
          # Traffic split: gradual rollout
          tag: ${{ github.sha }}
          no_traffic: true    # Deploy but don't send traffic yet

      - name: Gradual traffic migration
        run: |
          SERVICE="myapp"
          REGION="us-central1"
          NEW_TAG="${{ github.sha }}"

          # Send 10% traffic to new revision
          gcloud run services update-traffic $SERVICE \
            --region=$REGION \
            --to-tags=$NEW_TAG=10

          # Monitor error rate for 5 minutes
          sleep 300

          # Check error rate via Cloud Monitoring
          ERROR_RATE=$(gcloud logging read \
            "resource.type=cloud_run_revision AND \
             resource.labels.service_name=$SERVICE AND \
             httpRequest.status>=500" \
            --limit=100 \
            --format='value(httpRequest.status)' \
            --freshness=5m | wc -l)

          if [ "$ERROR_RATE" -lt 5 ]; then
            # Promote to 100%
            gcloud run services update-traffic $SERVICE \
              --region=$REGION \
              --to-latest
            echo "Deployed to 100% traffic"
          else
            echo "Error rate too high ($ERROR_RATE errors) -- rolling
back"
            gcloud run services update-traffic $SERVICE \
              --region=$REGION \
              --to-tags=stable=100
            exit 1
          fi
```

Azure Deployments

Azure Container Apps

```yaml
name: Deploy to Azure Container Apps

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  deploy-aca:
    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: azure/login@a457da9ea143d694b1b9c7c869ebb04edd5a2efb
        with:
          client-id:       ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id:       ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - name: Build and push to ACR
        run: |
          az acr login --name myregistry
          docker build -t myregistry.azurecr.io/myapp:${{ github.sha }} .
          docker push myregistry.azurecr.io/myapp:${{ github.sha }}

      - name: Deploy to Azure Container Apps
        uses: azure/container-apps-deploy-action@v1
        with:
          resourceGroup: my-resource-group
          containerAppName: myapp
          imageToDeploy: myregistry.azurecr.io/myapp:${{ github.sha }}
          containerAppEnvironment: my-environment
          targetPort: 8080
          ingress: external

      - name: Get deployment URL
        run: |
          URL=$(az containerapp show \
            --name myapp \
            --resource-group my-resource-group \
            --query 'properties.configuration.ingress.fqdn' \
            --output tsv)
          echo "Deployed to: https://$URL"
          echo "url=https://$URL" >> $GITHUB_STEP_SUMMARY
```

Interview Answer (30-45 seconds)

> "Cloud deployments in GitHub Actions follow a consistent three-step pattern: OIDC authentication to the cloud provider, build and push the container image to the provider's registry, then trigger the deployment service. For AWS ECS, the deployment uses `amazon-ecs-render-task-definition` to inject the new image into the task definition JSON, then `amazon-ecs-deploy-task-definition` to deploy and wait for service stability. For GCP Cloud Run, `deploy-cloudrun` handles the deployment with built-in support for traffic splitting — you can deploy a new revision with `no_traffic: true`, monitor it, then gradually shift traffic. For Lambda, it's a build-package-deploy pattern with `update-function-code` plus explicit version publishing and alias updates. In all cases the environment protection gate from GitHub Environments provides the approval mechanism, and OIDC eliminates stored credentials."

7.6 GitOps with ArgoCD — Image Updater, PR-Based Promotions

What It Is

GitOps is the pattern where the desired state of your infrastructure/deployments is stored in a git repository, and a reconciliation controller (ArgoCD) continuously applies that git state to your cluster. GitHub Actions' role shifts from "deploy directly" to "update the git state that ArgoCD reads."

The GitOps Mental Model

```
Traditional (Imperative) CI/CD:
  GitHub Actions: build image -> run kubectl apply / helm upgrade
  Problem: Actions has direct cluster write access
           State in cluster may drift from git
           No single source of truth for what's deployed

GitOps (Declarative):
  GitHub Actions: build image -> update image tag in git
  ArgoCD: continuously watches git -> applies to cluster
  Source of truth: git repo, always
  Actions never touches the cluster directly

  Trust boundaries:
    GitHub Actions: can write to git (low blast radius)
    ArgoCD: has cluster access (isolated, auditable)
    Cluster: never receives direct pushes from CI
```

Repository Structure for GitOps

```
my-org/
  app-repo/                        # Application code
    .github/workflows/ci.yml       # Build, test, push image
    src/
    Dockerfile

  gitops-repo/                     # Desired state (ArgoCD watches this)
    apps/
      payment-api/
        base/
          deployment.yaml
          service.yaml
          kustomization.yaml
        overlays/
          staging/
            kustomization.yaml     # image: ghcr.io/my-org/payment-
api:sha-abc123
          production/
            kustomization.yaml     # image: ghcr.io/my-org/payment-
api:sha-def456
    argocd/
      payment-api-staging.yaml     # ArgoCD Application resource
      payment-api-production.yaml
```

Pattern 1: GitHub Actions Updates GitOps Repo (Push-Based)

```yaml
# In app-repo/.github/workflows/ci-cd.yml
# After building and pushing image, update the gitops repo

name: CI/CD with GitOps

on:
  push:
    branches: [main]

permissions:
  contents: read
  packages: write
  id-token: write

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    outputs:
      image-tag: sha-${{ github.sha }}
      image-digest: ${{ steps.build.outputs.digest }}

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
```

```
      - uses: docker/setup-buildx-
action@b5730b5c3c4c89bdf3a737c8af1c4fc46a07e21d
      - uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - id: build
        uses: docker/build-push-
action@471d1dc4e07e5cdedd4c2171150001c434f0b7a4
        with:
          push: true
          tags: ghcr.io/my-org/payment-api:sha-${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  update-gitops-staging:
    needs: build-and-push
    runs-on: ubuntu-latest

    steps:
      # Checkout the GITOPS repo (not the app repo)
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          repository: my-org/gitops-repo
          token: ${{ secrets.GITOPS_PAT }}    # PAT with write access to
gitops-repo
          path: gitops-repo

      - name: Update staging image tag
        working-directory: gitops-repo
        run: |
          NEW_TAG="${{ needs.build-and-push.outputs.image-tag }}"

          # Update the kustomize image tag
          cd apps/payment-api/overlays/staging
          kustomize edit set image \
            ghcr.io/my-org/payment-api=ghcr.io/my-org/payment-
api:${NEW_TAG}

          # Verify the change
          cat kustomization.yaml

      - name: Commit and push to gitops repo
        working-directory: gitops-repo
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "github-
actions[bot]@users.noreply.github.com"
          git add apps/payment-api/overlays/staging/kustomization.yaml
          git commit -m "deploy(staging/payment-api): ${{ needs.build-and-
push.outputs.image-tag }}
```

```
          Deployed by: github.com/my-org/app-repo/actions/runs/${{
github.run_id }}
          Image digest: ${{ needs.build-and-push.outputs.image-digest }}"
          git push origin main
        env:
          GITHUB_TOKEN: ${{ secrets.GITOPS_PAT }}

  # ArgoCD will detect the gitops repo change and sync automatically
  # Wait for ArgoCD to confirm the sync
  wait-for-argocd-sync:
    needs: update-gitops-staging
    runs-on: ubuntu-latest
    steps:
      - name: Wait for ArgoCD sync
        run: |
          # Using ArgoCD CLI to check sync status
          # Requires: argocd CLI + argocd-server accessible from runner

          argocd app wait payment-api-staging \
            --sync \
            --health \
            --timeout 300 \
            --server argocd.internal.my-org.com \
            --auth-token ${{ secrets.ARGOCD_AUTH_TOKEN }}

          # Check final status
          argocd app get payment-api-staging \
            --server argocd.internal.my-org.com \
            --auth-token ${{ secrets.ARGOCD_AUTH_TOKEN }}
```

Pattern 2: PR-Based Production Promotion

```
# Promotion from staging to production requires a PR review
# GitHub Actions creates the PR; humans review and approve

  create-production-promotion-pr:
    needs: [build-and-push, wait-for-argocd-sync]
    runs-on: ubuntu-latest
    # Only create promotion PR on main branch builds
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          repository: my-org/gitops-repo
          token: ${{ secrets.GITOPS_PAT }}

      - name: Create promotion branch
        run: |
```

```
          NEW_TAG="${{ needs.build-and-push.outputs.image-tag }}"
          BRANCH="promote/payment-api-${NEW_TAG}"

          git checkout -b "$BRANCH"

          cd apps/payment-api/overlays/production
          kustomize edit set image \
            ghcr.io/my-org/payment-api=ghcr.io/my-org/payment-
api:${NEW_TAG}

          git config user.name "github-actions[bot]"
          git config user.email "github-
actions[bot]@users.noreply.github.com"
          git add kustomization.yaml
          git commit -m "promote(production/payment-api): ${NEW_TAG}

          Staging deployed: $(date -u +%Y-%m-%dT%H:%M:%SZ)
          Image digest: ${{ needs.build-and-push.outputs.image-digest }}
          App repo commit: ${{ github.sha }}"
          git push origin "$BRANCH"

          echo "BRANCH=$BRANCH" >> $GITHUB_ENV

      - name: Create promotion PR
        uses: actions/github-
script@60a0d83039c74a4aee543508d2ffcb1c3799cdea
        env:
          NEW_TAG: ${{ needs.build-and-push.outputs.image-tag }}
        with:
          github-token: ${{ secrets.GITOPS_PAT }}
          script: |
            const { data: pr } = await github.rest.pulls.create({
              owner: 'my-org',
              repo: 'gitops-repo',
              title: `promote(production/payment-api):
${process.env.NEW_TAG}`,
              body: `## Production Promotion: payment-api

              **Image:** \`ghcr.io/my-org/payment-
api:${process.env.NEW_TAG}\`
              **Staging deploy:** [Verified ✅](https://github.com/my-
org/app-repo/actions/runs/${{ github.run_id }})
              **App repo commit:** ${{ github.repository }}@${{ github.sha
}}

              ### Changes in this deployment
              ${{ github.event.head_commit.message }}

              ### Promotion checklist
              - [ ] Staging smoke tests passed
              - [ ] No errors in staging for 30+ minutes
              - [ ] Change reviewed by on-call engineer

              _This PR was created automatically. Merge to deploy to
```

```
production._`,
              head: process.env.BRANCH,
              base: 'main'
            });

            console.log(`Created PR #${pr.number}: ${pr.html_url}`);

            // Add reviewers
            await github.rest.pulls.requestReviewers({
              owner: 'my-org',
              repo: 'gitops-repo',
              pull_number: pr.number,
              team_reviewers: ['platform-engineers']
            });
```

ArgoCD Application Resource (in gitops-repo)

```
# gitops-repo/argocd/payment-api-staging.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: payment-api-staging
  namespace: argocd
  annotations:
    notifications.argoproj.io/subscribe.on-sync-succeeded.slack:
deployments-staging
    notifications.argoproj.io/subscribe.on-health-degraded.slack: alerts-
critical
spec:
  project: default
  source:
    repoURL: https://github.com/my-org/gitops-repo
    targetRevision: HEAD
    path: apps/payment-api/overlays/staging
  destination:
    server: https://kubernetes.default.svc
    namespace: staging
  syncPolicy:
    automated:
      prune: true          # Delete resources removed from git
      selfHeal: true       # Re-apply if cluster state drifts
      allowEmpty: false    # Don't sync if manifests are empty
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true
    retry:
      limit: 3
      backoff:
        duration: 5s
        maxDuration: 3m
```

```yaml
        factor: 2
---
# Production application requires manual sync (no automated sync)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: payment-api-production
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/my-org/gitops-repo
    targetRevision: HEAD
    path: apps/payment-api/overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated: null    # MANUAL SYNC only for production
    syncOptions:
      - ServerSideApply=true
```

Interview Answer (30-45 seconds)

> "In a GitOps model with ArgoCD, GitHub Actions doesn't deploy to Kubernetes directly — it updates image tags in a separate git repository that ArgoCD watches. The workflow is: build and push the image, then open a separate checkout of the gitops repo, run `kustomize edit set image` to update the tag, and push the change. ArgoCD detects the git change and syncs the cluster. For production promotion, the workflow creates a PR in the gitops repo with the new image tag — a human reviews and merges it, triggering ArgoCD to sync production. This model has three key benefits: the cluster state is always exactly what's in git, GitHub Actions never needs direct cluster credentials, and every production change is a git commit with a full audit trail. The tradeoff: the feedback loop is longer — you wait for ArgoCD to sync rather than running `kubectl` directly."

Gotchas ⚠

- **A PAT is required to push to the gitops repo.** `GITHUB_TOKEN` is scoped to the current repo — it cannot write to another repo. Use a `GITOPS_PAT` (fine-grained PAT with only `contents: write` on the gitops repo) or a GitHub App token.
- **Race conditions on concurrent image tag updates.** If two services build simultaneously and both try to push to the gitops repo, one push will be rejected with "non-fast-forward." Add retry logic with exponential backoff and `git pull --rebase` before retrying the push.
- **ArgoCD sync does not mean deployment success.** ArgoCD "Synced" means manifests were applied. "Healthy" means the pods passed readiness probes. Always wait for both conditions. The `argocd app wait --health --sync` command handles this.
- **`kustomize edit set image` modifies the `kustomization.yaml` in place** with a specific format. If your kustomization.yaml uses a different image format (digest-based instead of tag-based),

the edit command may not find the right entry.

7.7 Blue-Green and Canary Deployment Patterns

What It Is

Traffic management strategies that minimize risk during deployments by controlling how traffic shifts from the old version to the new version. Blue-green does an instantaneous switch between two identical environments. Canary gradually shifts a percentage of traffic to the new version before full rollout.

Blue-Green Deployment

```
BLUE (current live):  v1.0  <-- 100% of traffic
GREEN (new version):  v1.1  <-- 0% of traffic (deployed, not live)

Step 1: Deploy v1.1 to GREEN environment (live traffic unaffected)
Step 2: Run smoke tests against GREEN
Step 3: Switch load balancer: GREEN becomes live (100% traffic)
Step 4: Keep BLUE running for 10-30 minutes (quick rollback available)
Step 5: If healthy, decommission BLUE (or it becomes the new STANDBY)
```

Blue-Green on Kubernetes with Nginx Ingress

```yaml
name: Blue-Green Deployment

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  blue-green-deploy:
    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
```

```
          role-to-assume: ${{ secrets.PROD_DEPLOY_ROLE_ARN }}
          aws-region: us-east-1

      - name: Configure kubectl
        run: aws eks update-kubeconfig --name prod-cluster --region us-
east-1

      # Determine which slot is currently active
      - name: Determine active slot
        id: slots
        run: |
          ACTIVE=$(kubectl get service myapp \
            --namespace production \
            -o jsonpath='{.spec.selector.slot}' 2>/dev/null || echo
"blue")
          echo "active=${ACTIVE}" >> $GITHUB_OUTPUT
          echo "inactive=$([[ '$ACTIVE' == 'blue' ]] && echo 'green' ||
echo 'blue')" >> $GITHUB_OUTPUT
          echo "Currently active: $ACTIVE -- deploying to: $([[ '$ACTIVE'
== 'blue' ]] && echo 'green' || echo 'blue')"

      # Deploy to the INACTIVE slot (no user traffic)
      - name: Deploy to inactive slot
        run: |
          SLOT="${{ steps.slots.outputs.inactive }}"
          IMAGE="ghcr.io/my-org/myapp:sha-${{ github.sha }}"

          kubectl set image deployment/myapp-${SLOT} \
            app=${IMAGE} \
            --namespace production

          kubectl rollout status deployment/myapp-${SLOT} \
            --namespace production \
            --timeout=10m

          echo "Deployed $IMAGE to $SLOT slot"

      # Smoke test the inactive slot directly (bypass main service)
      - name: Smoke test inactive slot
        run: |
          SLOT="${{ steps.slots.outputs.inactive }}"

          # Create temporary port-forward to inactive slot service
          kubectl port-forward service/myapp-${SLOT} 8080:80 \
            --namespace production &
          PF_PID=$!
          sleep 3

          # Run smoke tests
          TESTS_PASSED=true
          for endpoint in /health /api/v1/status; do
            STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
              "http://localhost:8080${endpoint}" --max-time 5)
            echo "Test $endpoint: HTTP $STATUS"
```

```
            [ "$STATUS" = "200" ] || TESTS_PASSED=false
          done

          kill $PF_PID 2>/dev/null || true

          if [ "$TESTS_PASSED" != "true" ]; then
            echo "Smoke tests failed -- aborting traffic switch"
            exit 1
          fi
          echo "All smoke tests passed"

      # Switch traffic to the new slot
      - name: Switch traffic to new slot
        id: switch
        run: |
          INACTIVE="${{ steps.slots.outputs.inactive }}"

          # Patch the main service selector to point to new slot
          kubectl patch service myapp \
            --namespace production \
            --type='json' \
            -p="
[{\"op\":\"replace\",\"path\":\"/spec/selector/slot\",\"value\":\"${INACTI
VE}\"}]"

          echo "Traffic switched to: $INACTIVE"
          echo "new-active=${INACTIVE}" >> $GITHUB_OUTPUT

      # Monitor for 5 minutes post-switch
      - name: Post-switch monitoring
        run: |
          echo "Monitoring new deployment for 5 minutes..."
          FAILURES=0
          for i in $(seq 1 30); do
            STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
              https://app.my-org.com/health --max-time 5)

            if [ "$STATUS" = "200" ]; then
              echo "Check $i/30: OK"
            else
              FAILURES=$((FAILURES + 1))
              echo "Check $i/30: FAILED (HTTP $STATUS)"
            fi

            [ $FAILURES -gt 3 ] && echo "Too many failures -- triggering
rollback" && exit 1
            sleep 10
          done
          echo "Monitoring complete: deployment healthy"

      # Rollback if monitoring fails
      - name: Rollback on failure
        if: failure() && steps.switch.outcome == 'success'
        run: |
```

```
          OLD_ACTIVE="${{ steps.slots.outputs.active }}"
          echo "Rolling back to: $OLD_ACTIVE"

          kubectl patch service myapp \
            --namespace production \
            --type='json' \
            -p="
[{\"op\":\"replace\",\"path\":\"/spec/selector/slot\",\"value\":\"${OLD_AC
TIVE}\"}]"

          echo "Rollback complete -- traffic restored to $OLD_ACTIVE"

      - name: Write deployment summary
        if: always()
        run: |
          cat >> $GITHUB_STEP_SUMMARY << EOF
          ## Blue-Green Deployment Summary

          | Field | Value |
          |-------|-------|
          | Active slot | ${{ steps.slots.outputs.inactive }} |
          | Previous slot | ${{ steps.slots.outputs.active }} |
          | Image | \`sha-${{ github.sha }}\` |
          | Status | ${{ job.status }} |
          EOF
```

Canary Deployment

```
v1.0: 100% traffic  -- Stable version
                            |
v1.1 deployed (canary)      |
v1.0: 90% traffic           |   Phase 1: Observe 5 minutes
v1.1: 10% traffic     ------+
                            |
v1.0: 50% traffic           |   Phase 2: If error rate < 1%, expand to 50%
v1.1: 50% traffic     ------+   Observe 10 minutes
                            |
v1.0:  0% traffic           |   Phase 3: If still healthy, 100%
v1.1: 100% traffic    ------+   Decommission v1.0
```

Canary with Argo Rollouts

```yaml
# ArgoRollout resource (deploy once, managed by Argo Rollouts controller)
# rollout.yaml -- in gitops repo
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: myapp
```

```yaml
  namespace: production
spec:
  replicas: 10
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: ghcr.io/my-org/myapp:latest  # Updated by CI
          ports:
            - containerPort: 8080
  strategy:
    canary:
      # Traffic management via Nginx Ingress
      trafficRouting:
        nginx:
          stableIngress: myapp-stable
      steps:
        - setWeight: 10      # 10% traffic
        - pause: {duration: 5m}   # Wait 5 minutes
        - setWeight: 25
        - pause: {duration: 5m}
        - setWeight: 50
        - pause: {duration: 10m}
        - analysis:           # Run analysis at 50%
            templates:
              - templateName: success-rate
        - setWeight: 100      # Full rollout
      autoPromotionEnabled: false   # Require manual approval to proceed
past analysis
      analysis:
        templates:
          - templateName: success-rate
        args:
          - name: service-name
            value: myapp-canary
```

```yaml
# GitHub Actions workflow that triggers the Argo Rollout
name: Canary Deploy via Argo Rollouts

on:
  push:
    branches: [main]

permissions:
  id-token: write
```

```
    contents: read

  jobs:
    canary-deploy:
      runs-on: ubuntu-latest
      environment: production

      steps:
        - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

        - uses: aws-actions/configure-aws-
  credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
          with:
            role-to-assume: ${{ secrets.PROD_DEPLOY_ROLE_ARN }}
            aws-region: us-east-1

        - name: Configure kubectl
          run: aws eks update-kubeconfig --name prod-cluster --region us-
  east-1

        - name: Install Argo Rollouts kubectl plugin
          run: |
            curl -LO https://github.com/argoproj/argo-
  rollouts/releases/latest/download/kubectl-argo-rollouts-linux-amd64
            chmod +x kubectl-argo-rollouts-linux-amd64
            mv kubectl-argo-rollouts-linux-amd64 /usr/local/bin/kubectl-
  argo-rollouts

        - name: Update rollout image (triggers canary)
          run: |
            kubectl argo rollouts set image myapp \
              myapp=ghcr.io/my-org/myapp:sha-${{ github.sha }} \
              --namespace production

            echo "Canary rollout started"

        - name: Monitor canary rollout
          run: |
            # Watch rollout status for up to 45 minutes
            TIMEOUT=2700
            START=$(date +%s)

            while true; do
              NOW=$(date +%s)
              ELAPSED=$((NOW - START))

              [ $ELAPSED -gt $TIMEOUT ] && echo "Canary timed out" && exit 1

              STATUS=$(kubectl argo rollouts get rollout myapp \
                --namespace production \
                -o json | jq -r '.status.phase')

              echo "$(date): Rollout phase: $STATUS"
```

```
            case "$STATUS" in
              Healthy)
                echo "Rollout complete and healthy"
                exit 0
                ;;
              Degraded)
                echo "Rollout degraded -- check Argo Rollouts for details"
                kubectl argo rollouts get rollout myapp --namespace
production
                exit 1
                ;;
              Paused)
                echo "Rollout paused -- waiting for manual promotion or
analysis"
                ;;
            esac

            sleep 30
          done

      - name: Abort canary on workflow failure
        if: failure()
        run: |
          kubectl argo rollouts abort myapp --namespace production
          kubectl argo rollouts undo myapp --namespace production
          echo "Canary aborted and rolled back"
```

Canary with Native Kubernetes (No Argo Rollouts)

```
# Manual canary using separate deployments + traffic weighting via replica
count
# 10% canary = 1 canary pod + 9 stable pods (rough approximation)

jobs:
  manual-canary:
    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Configure kubectl
        run: aws eks update-kubeconfig --name prod-cluster --region us-
east-1

      - name: Deploy canary (1 pod = ~10%)
        run: |
          # Create/update the canary deployment with new image
          kubectl set image deployment/myapp-canary \
            app=ghcr.io/my-org/myapp:sha-${{ github.sha }} \
            --namespace production
```

```
          kubectl scale deployment/myapp-canary \
            --replicas=1 \
            --namespace production

          kubectl rollout status deployment/myapp-canary \
            --namespace production \
            --timeout=5m

      - name: Observe canary (10 minutes)
        run: |
          echo "Observing canary for 10 minutes..."
          sleep 600

          # Check canary pod error rate via metrics
          ERROR_RATE=$(kubectl exec -n monitoring prometheus-0 -- \
            promtool query instant \
            'rate(http_requests_total{job="myapp-canary",status=~"5.."}
[5m]) /
             rate(http_requests_total{job="myapp-canary"}[5m])' \
            2>/dev/null | grep -oP '\d+\.\d+' | head -1 || echo "0")

          echo "Canary error rate: $ERROR_RATE"
          THRESHOLD="0.01"
          if (( $(echo "$ERROR_RATE > $THRESHOLD" | bc -l) )); then
            echo "Error rate $ERROR_RATE exceeds threshold $THRESHOLD --
rolling back"
            kubectl scale deployment/myapp-canary --replicas=0 --namespace
production
            exit 1
          fi

      - name: Promote canary to stable
        run: |
          # Update stable deployment with new image
          kubectl set image deployment/myapp-stable \
            app=ghcr.io/my-org/myapp:sha-${{ github.sha }} \
            --namespace production

          kubectl rollout status deployment/myapp-stable \
            --namespace production \
            --timeout=10m

          # Scale down canary (stable now handles all traffic)
          kubectl scale deployment/myapp-canary \
            --replicas=0 \
            --namespace production

          echo "Canary promoted to stable"
```

Interview Answer (30-45 seconds)

> "Blue-green and canary are risk-reduction strategies for deployments. Blue-green maintains two identical environments — blue is live, green receives the new deployment, smoke tests run against green, then the load balancer switches in one operation. Rollback is instantaneous: flip the switch back. GitHub Actions implements this by checking which slot is active, deploying to the inactive slot, smoke testing it directly, then patching the Kubernetes Service selector. Canary is more gradual — 10% of traffic goes to the new version, you observe error rates and latency for 5-10 minutes, then expand to 25%, 50%, and finally 100%. Argo Rollouts manages canary traffic splitting via Ingress annotations automatically; without it, you simulate canary via replica count ratios. The GitHub Actions role in both patterns is to trigger the deployment mechanism and monitor health metrics, with automated rollback triggered by failure conditions."

Gotchas ⚠

- **Blue-green double compute cost.** During the switch window, you're running both blue AND green at full capacity. For memory/CPU-heavy services, this can briefly 2x your infrastructure cost. Size your cluster to absorb this.
- **Database schema migrations break blue-green.** If v1.1 requires a schema change that v1.0 is incompatible with, you can't run both simultaneously. Solution: expand-contract pattern — make the schema backward-compatible first, deploy, then remove the compatibility shim in a follow-up.
- **Canary traffic splitting via replica count is imprecise.** 1 canary pod + 9 stable pods is roughly 10% canary, but Kubernetes load balancing isn't perfectly proportional. For accurate traffic percentages, use a proper traffic management layer (Nginx Ingress, Istio, Argo Rollouts).
- **Session affinity breaks canary observation.** If sticky sessions route a user always to the same pod, your canary error rate is skewed — the same users always hit the canary. Ensure no session affinity during canary evaluation, or measure at the request level, not user level.
- **Argo Rollouts `pause: {}` with no duration requires manual promotion.** `pause: {duration: 5m}` auto-proceeds after 5 minutes. `pause: {}` with no duration blocks forever until `kubectl argo rollouts promote`. This is intentional for gates but surprises people who expect it to auto-advance.

Cross-Topic Interview Questions

Q: Walk through a complete CI/CD pipeline for a new microservice from code push to production, including all security controls.

> "Push triggers the CI workflow. First: lint, test, coverage check — on GitHub-hosted runner (no secrets needed, safe for PRs from forks). Second: security scan — Trivy filesystem scan with SARIF upload to GitHub Security tab, secret scanning, dependency review action on the PR. Third: Docker build with BuildKit — multi-arch (AMD64 + ARM64), layer cache via `type=gha`, BuildKit secret mounts for any private registry auth, SBOM generation and SLSA provenance attestation attached to the image digest. Push to GHCR with GITHUB_TOKEN — no separate registry credentials.
>
> On merge to main: update the image tag in the gitops repo via a PR. The gitops PR runs a diff preview via ArgoCD dry-run as a PR check. A platform engineer reviews and approves the gitops PR.

> Merge triggers ArgoCD sync to staging. ArgoCD syncs, runs health checks, notifies Slack. Automated smoke tests run against staging. A promotion PR is auto-created for production. On-call engineer reviews, approves, ArgoCD syncs production with a canary strategy via Argo Rollouts — 10% for 5 minutes, analysis runs, then promotes to 100%. GitHub Environment protection gates ensure OIDC tokens are only issued to the production environment's deploy workflow. No long-lived credentials anywhere."

Q: You're using GHCR for your container images. A developer reports that a PR from a forked repository fails to pull the base image in the Dockerfile FROM ghcr.io/my-org/private-image:latest. Why and how do you fix it?

> "Fork PRs run with `pull_request` trigger and no access to repository secrets — GITHUB_TOKEN on fork PRs has read-only permissions on the calling repo but cannot authenticate to pull a private GHCR image. The build fails at the Dockerfile FROM stage because the runner has no credentials for `ghcr.io/my-org/private-image`. Three solutions. Easiest: make the base image public — if it's just a build environment image with no sensitive content, public GHCR is fine. Second: use a workflow-level pre-pull with an org-level PAT stored as a repo secret, but fork PRs don't have secret access, so this doesn't work for the fork scenario. Correct solution for forks: use `pull_request_target` only for the image pull and build steps, NOT for executing the forked code — but this is dangerous (see category 4). The safest pattern: build without private base images for fork PRs by using a public equivalent or maintaining a public builder image, and only use the private base image for merges to main where secrets are available."

Q: Your Terraform apply workflow runs on PR merge but sometimes applies a different plan than what was shown in the PR. How do you prevent this?

> "This is the time-of-check/time-of-use (TOCTOU) problem in Terraform workflows. Between the `terraform plan` on the PR and `terraform apply` after merge, the infrastructure state or provider state may have changed — another PR merged, someone applied manually, or a cloud resource changed. The fix is three-pronged. One: save the plan binary as an artifact in the plan job (`-out=tfplan`) and download that exact artifact in the apply job — never re-run plan in the apply job. Two: add a final `terraform plan -detailed-exitcode` immediately before apply as a drift check — if exit code is 2 (changes differ from saved plan), fail the apply and require a new PR. Three: use `concurrency: group: terraform-apply-${{ github.repository }} cancel-in-progress: false` to serialize applies — never run two applies simultaneously to prevent state contention. Additionally, restrict who can merge to main for Terraform-containing paths via CODEOWNERS."

Quick Reference Cheat Sheet

Docker Build Essentials

```
# Full build with all features
- uses: docker/setup-qemu-action@<SHA>       # Multi-arch emulation
```

```yaml
  - uses: docker/setup-buildx-action@<SHA>     # BuildKit builder
  - uses: docker/login-action@<SHA>            # Registry auth
    with:
      registry: ghcr.io
      username: ${{ github.actor }}
      password: ${{ secrets.GITHUB_TOKEN }}
  - uses: docker/metadata-action@<SHA>         # Generate tags
    id: meta
    with:
      images: ghcr.io/${{ github.repository }}
  - uses: docker/build-push-action@<SHA>
    with:
      platforms: linux/amd64,linux/arm64
      push: ${{ github.event_name != 'pull_request' }}
      tags: ${{ steps.meta.outputs.tags }}
      cache-from: type=gha
      cache-to: type=gha,mode=max
      provenance: mode=max
      sbom: true
```

GHCR Naming

```
ghcr.io/<github-owner>/<image-name>:<tag>
ghcr.io/${{ github.repository_owner }}/myapp:sha-${{ github.sha }}
ghcr.io/${{ github.repository }}:<tag>  # org/repo as image name
```

Kubernetes Deploy Checklist

```
[ ] OIDC auth to cloud provider (no kubeconfig in secrets)
[ ] Use Helm --atomic for auto-rollback on failure
[ ] kubectl rollout status --timeout to verify
[ ] HTTP health probe after rollout
[ ] kubectl dump logs if rollout fails
[ ] concurrency: to prevent concurrent deploys
[ ] environment: for approval gate
```

Terraform Workflow Rules

```
[ ] SEPARATE roles for plan (read) and apply (write)
[ ] Save plan as artifact; apply THAT exact artifact
[ ] -detailed-exitcode to detect "no changes" vs "error"
[ ] Post plan to PR comment (update, don't create new)
[ ] concurrency: to serialize applies
[ ] Drift detection cron + GitHub Issue creation
[ ] Only apply if needs.plan.outputs.exitcode == '2'
```

Deployment Strategy Selection

```
Low risk change, fast rollback:  Blue-Green
Gradual confidence building:     Canary
GitOps with ArgoCD:              Argo Rollouts (preferred)
Simple services:                 Helm --atomic
Serverless:                      Lambda alias + gradual traffic shifting
```

GitOps PR Flow

```
app-repo push -> build image -> push to GHCR
             -> checkout gitops-repo (needs PAT, not GITHUB_TOKEN)
             -> kustomize edit set image
             -> push change to gitops-repo
             -> ArgoCD detects change -> syncs staging
             -> wait for ArgoCD health
             -> create PR in gitops-repo for production promotion
             -> human reviews and merges
             -> ArgoCD syncs production (manual or canary)
```

End of Category 7: CI/CD for Real Workloads

> **Next:** Category 8 (Monorepo Patterns) extends the dynamic matrix, path filtering, and selective build concepts introduced here — specifically the challenge of detecting affected services, managing cross-service dependencies, and integrating tools like Turborepo and Nx into GitHub Actions workflows.
>
> **Priority drill topics from this category:**
>
> - Docker buildx: explain the QEMU vs native build tradeoff for compiled languages; write the complete four-action sequence from memory
> - Terraform TOCTOU: explain the plan artifact pattern and why re-running plan in the apply job is wrong
> - GitOps promotion: describe the gitops repo update pattern including why GITHUB_TOKEN won't work and the PAT requirement
> - Blue-green rollback: explain the Service selector switch mechanism and the database migration incompatibility problem